

OS Lab5 实验报告

组员：查科言(2312189) 禹相祐(2312900) 董丰瑞(2311973)

分工：

1. 练习1~3: 查科言
2. Challenge 1-2: 禹相祐
3. GDB调试: 董丰瑞
4. 报告撰写: 三人一起

练习1：加载应用程序并执行

load_icode 填写 (Exercise 1 相关)

在 `load_icode` 的第6步，设置 `trapframe`：

```
// 核心设置：配置用户态执行的关键上下文
// (1) 设置用户栈指针 (sp)：指向用户栈顶
tf->gpr.sp = USTACKTOP;
// (2) 设置用户态入口地址 (epc)：ELF的入口地址（第一条指令地址）
tf->epc = elf->e_entry;
// (3) 设置 sstatus 寄存器：配置用户态执行的权限和中断状态
tf->status = (sstatus & ~(SSTATUS_SPP | SSTATUS_SIE)) | SSTATUS_SPIE;
```

在操作系统加载用户程序的过程中，`load_icode` 函数负责把ELF格式的二进制文件转换成可执行的用户进程。其中，最后设置 `trapframe`（陷阱帧）的三行代码让CPU能准确、安全地从内核态切换到用户态，开始执行新程序。下面我们就来分析下为什么这么写这三行。

第一行：`tf->gpr.sp = USTACKTOP`

——给用户程序“准备好工作台”

作用：设置用户栈的起始位置

`sp` 是CPU里的“栈指针寄存器”，它指向当前程序正在使用的栈的顶部。栈对于程序来说就像一个“临时工作台”，比如函数调用时传递参数、保存局部变量，都需要用到栈。

为什么这么写？

- `USTACKTOP` 是内核提前规定好的“用户栈顶部地址”（比如在 `ucore` 中，通常是一个固定的高地址）。
- 用户程序的栈是“向下生长”的（从高地址往低地址用），所以一开始要让 `sp` 指向栈的最顶端，这样程序运行时往栈里存数据，`sp` 会自动往下移，不会乱占其他内存。
- 如果不设置 `sp`，用户程序一运行就会找不到栈，执行函数时就会“无处安放”临时数据，直接崩溃。

第二行: `tf->epc = elf->e_entry`

——告诉CPU“程序从这里开始跑”

作用：设置用户程序的入口地址

`epc` 是CPU里的“异常返回地址寄存器”。当内核态通过 `sret` 指令（从异常中返回）切换到用户态时，CPU会自动跳转到 `epc` 指向的地址，开始执行那里的指令。

为什么这么写？

- `elf->e_entry` 是ELF文件里记录的“程序入口地址”——也就是用户程序编译后，第一条指令所在的位置。
- 比如我们写的 `main` 函数，最终会被编译成机器码，而 `e_entry` 就指向 `main` 函数对应的第一条机器指令。
- 如果这里填错了地址，CPU就会跑到一个随机的位置执行指令，可能是无效数据，程序肯定会出错。

第三行: `tf->status = (sstatus & ~(SSTATUS_SPP | SSTATUS_SIE)) | SSTATUS_SPIE`

——给CPU“切换模式”，确保安全运行

作用：设置CPU的运行状态（权限和中断）

`status` 对应CPU的 `sstatus` 寄存器，它像一个“模式开关”，控制CPU是运行在用户态还是内核态，以及是否允许接收中断。

为什么这么写？

这行代码看起来复杂，拆开来理解其实很简单：

1. `sstatus & ~(SSTATUS_SPP | SSTATUS_SIE)`：清除两个关键标志
 - `SSTATUS_SPP`：表示“上一个特权级”。内核态运行时这个标志是1，现在要切换到用户态，所以把它清0，告诉CPU“接下来是用户态”。
 - `SSTATUS_SIE`：表示“全局中断使能”。内核态可能为了安全关闭了中断，这里先保持关闭，避免切换过程中被打断。
2. `| SSTATUS_SPIE`：设置“保存的中断使能”标志
 - `SSTATUS_SPIE`：表示“切换到用户态后，允许接收中断”。用户程序运行时需要响应键盘输入、时钟等中断，所以这里提前打开权限。

总结：这三行代码的核心意义

用户程序从“二进制文件”变成“可执行进程”，最后一步就是以下三行代码：

- 栈指针（`sp`）保证程序能正常使用临时数据；
- 入口地址（`epc`）让CPU知道从哪条指令开始执行；
- 状态寄存器（`status`）让CPU安全切换到用户态并允许中断。

练习1 思考题：简要描述用户态进程被ucore选择占用CPU执行（RUNNING态）到具体执行应用程序第一条指令的整个经过

阶段 1：调度器选中进程，标记为 RUNNING 态

1. ucore 调度器（schedule 函数）遍历就绪队列，根据调度策略（如时间片轮转）选中该进程；
2. 调用 proc_run 函数触发进程上下文切换：
 - 禁用中断（local_intr_save），保证切换过程原子性；
 - 更新全局 current 指针为该进程，标记进程状态为 PROC_RUNNING；
 - 加载该进程的页目录表物理地址到 satp 寄存器（lsatp(current->pgdir)），确保 CPU 使用该进程的页表解析虚拟地址。

阶段 2：内核态恢复进程上下文，准备切换到用户态

1. proc_run 调用 switch_to 函数，恢复该进程的内核上下文（context 结构体）：
 - context 保存了进程上次在内核态暂停时的 CPU 寄存器状态（如 ra、sp 等）；
 - switch_to 通过栈切换，将 CPU 寄存器恢复为该进程的内核上下文，跳转到该进程上次暂停的位置（即 do_execve 系统调用的返回路径）。

阶段 3：执行 sret 指令，完成内核态→用户态特权级切换

1. 进程在内核态执行到 do_execve 系统调用的返回逻辑，最终触发 sret 指令（RISC-V 异常返回指令）；
2. sret 指令读取 trapframe 中的关键字段，完成硬件上下文配置：
 - 从 tf->status 加载 sstatus 寄存器：
 - SPP=0：确认返回后进入用户态（U-mode）；
 - SPIE=1：开启用户态中断；
 - 从 tf->epc 加载 sepc 寄存器：将程序计数器（pc）设置为应用程序入口地址（elf->e_entry）；
 - 从 tf->gpr.sp 加载用户态栈指针 sp：指向 USTACKTOP（用户栈顶）。

阶段 4：地址映射与执行第一条指令

1. CPU 切换到用户态后，解析 sepc 中的入口地址（elf->e_entry，用户虚拟地址）：
 - 通过 satp 寄存器指向的页目录表，逐层查找页表项（PTE），将虚拟地址转换为物理地址（即 load_icode 中拷贝代码段的物理页）；
2. CPU 从该物理地址读取应用程序的第一条机器指令，完成指令译码、执行；
3. 用户态栈指针 sp 指向 USTACKTOP，保证程序启动时能正常使用栈（如执行 _start 函数、初始化 libc、调用 main 函数）。

关键：调度器选中进程（RUNNING）→ proc_run 切换页表/上下文 → switch_to 恢复内核上下文 → sret 读取 trapframe 切换到用户态 → 地址映射 → 执行第一条指令

练习2：父进程复制内存空间给子进程

补充完后的 copy_range 函数源码

```
// 实现 fork 时父子进程用户内存复制 的最终落地函数，核心作用是以物理页为单位，
// 将父进程（进程 A）用户地址空间 [start, end) 范围内的内存数据，完整复制到子进程（进程 B）的
// 地址空间中。
// do_fork（创建子进程）→ copy_mm（复制内存管理结构）→ dup_mmap（复制虚拟内存区域 VMA）→
// copy_range（复制实际物理页数据）
int copy_range(pde_t *to, pde_t *from, uintptr_t start, uintptr_t end,
```

```

        bool share)
{
    assert(start % PGSIZE == 0 && end % PGSIZE == 0);
    assert(USER_ACCESS(start, end)); // 判断地址是否在用户态可访问范围
    // copy content by page unit.
    do // 按页遍历复制
    {
        // 根据父进程页目录表 (from) 和虚拟地址 (start), 查找对应的页表项 (PTE) 的内核虚拟
地址
        pte_t *ptep = get_pte(from, start, 0), *nptep;
        // 父进程该虚拟地址范围的二级页表未分配 (无有效数据), 无需复制, 跳过该页表范围
        (+PTSIZE)
        if (ptep == NULL)
        {
            start = ROUNDDOWN(start + PTSIZE, PTSIZE);
            continue;
        }
        // call get_pte to find process B's pte according to the addr start. If
        // pte is NULL, just alloc a PT
        if (*ptep & PTE_V) // PTE_V: 页表项的「有效位」, 值为 1 表示该虚拟地址对应物理页已
分配 (父进程该页有有效数据); 值为 0 则无数据, 无需复制
        {
            if ((nptep = get_pte(to, start, 1)) == NULL) // get_pte(to, start,
1):根据子进程页目录表 (to) 和虚拟地址 (start), 查找 / 创建对应的页表项
            {
                return -E_NO_MEM; // 子进程页表项创建失败, 返回内存不足错误
            }
            uint32_t perm = (*ptep & PTE_USER); // 提取父进程该页的用户权限, 保证子进程
页的访问权限与父进程完全一致
            // get page from ptep
            struct Page *page = pte2page(*ptep);
            // 为子进程分配新的空闲物理页
            struct Page *npage = alloc_page();
            assert(page != NULL);
            assert(npage != NULL);
            int ret = 0;
            // 转换为内核虚拟地址, 进行内存复制
            void *src_kvaddr = page2kva(page);
            void *dst_kvaddr = page2kva(npage);
            // 拷贝整页数据
            memcpy(dst_kvaddr, src_kvaddr, PGSIZE);
            // 将子进程物理页插入页表 (关联虚拟地址)
            ret = page_insert(to, npage, start, perm);
            if (ret != 0)
            {
                free_page(npage);
                return ret;
            }
        }
        start += PGSIZE; // 继续处理下一个页
    } while (start != 0 && start < end);
    return 0;
}

```

详细分析

`copy_range` 函数是 ucore 操作系统中实现父子进程用户态内存复制的核心函数，用于在 `fork` 过程中将父进程（进程A）用户地址空间 `[start, end)` 范围内的内存以物理页为单位完整复制到子进程（进程B），确保两者虚拟地址内容一致但物理页独立。其详细逻辑可拆解为以下关键环节：

1. 入参合法性校验：确保复制范围安全有效

- **地址对齐检查**：通过 `assert(start % PGSIZE == 0 && end % PGSIZE == 0)` 确保 `start` 和 `end` 均为物理页大小（`PGSIZE`，通常为4KB）的整数倍。这是因为内存复制以物理页为最小单位，非对齐地址会导致复制不完整或越界。
- **用户态范围检查**：通过 `assert(USER_ACCESS(start, end))` 验证 `[start, end)` 地址范围属于用户态可访问空间（而非内核态地址），防止越界访问内核内存，保证系统安全性。

2. 核心循环：按物理页遍历并复制父进程内存

函数通过 `do-while` 循环逐页处理 `[start, end)` 范围内的内存，每一轮循环处理一个物理页，具体步骤如下：

步骤2.1：查找父进程对应虚拟地址的页表项（PTE）

- 调用 `get_pte(from, start, 0)` 从父进程的页目录表（`from`）中查找虚拟地址 `start` 对应的页表项（PTE）。第三个参数 `0` 表示“仅查找，不分配”——若父进程在该虚拟地址范围内没有有效页表（即无数据），则返回 `NULL`。

步骤2.2：处理父进程无有效页表的情况

- 若 `ptep == NULL`（父进程该页表范围无有效数据），则直接跳过整个页表覆盖的虚拟地址范围（`PTSIZE` 为一个页表可映射的虚拟地址大小，通常为4MB），通过 `start = ROUNDDOWN(start + PTSIZE, PTSIZE)` 将 `start` 跳至下一个页表的起始地址，减少无效遍历，提高效率。

步骤2.3：处理父进程有有效数据的页（PTE有效）

当父进程的页表项有效（`*ptep & PTE_V` 为真，`PTE_V` 是页表项的“有效位”），说明该虚拟地址对应物理页已分配且有数据，需要复制到子进程：

- **为子进程创建页表项**：调用 `get_pte(to, start, 1)` 为子进程（`to` 是子进程页目录）查找或创建 `start` 对应的页表项。第三个参数 `1` 表示“若不存在则分配新页表”，若分配失败（返回 `NULL`），函数返回 `-E_NO_MEM`（内存不足错误）。
- **继承父进程的页权限**：通过 `uint32_t perm = (*ptep & PTE_USER)` 提取父进程该页的用户态权限（如读写、执行等），确保子进程的内存访问权限与父进程完全一致。
- **获取父进程的物理页**：通过 `pte2page(*ptep)` 将父进程的页表项（PTE）转换为对应的物理页结构体（`struct Page`），用于后续数据复制。
- **为子进程分配新物理页**：调用 `alloc_page()` 为子进程分配一块全新的空闲物理页（`npage`），并通过 `assert` 确保分配成功（物理页非空）。
- **复制页数据**：由于物理页无法直接通过虚拟地址访问，需先通过 `page2kva` 将物理页转换为内核虚拟地址（内核映射的物理页地址），再用 `memcpy(dst_kvaddr, src_kvaddr, PGSIZE)` 复制整页数据（4KB），确保子进程物理页内容与父进程完全一致。
- **关联子进程物理页与虚拟地址**：调用 `page_insert(to, npage, start, perm)` 将子进程的新物理页（`npage`）与虚拟地址 `start` 关联，并设置权限（`perm`），完成子进程页表的更新。若关联失败，释放已分配的物理页（`free_page(npage)`）并返回错误码。

3. 循环推进与终止

每处理完一个物理页后，通过 `start += PGSIZE` 推进到下一个物理页，循环持续到 `start` 达到 `end`（即覆盖整个 `[start, end)` 范围），最终返回 `0` 表示复制成功。

总的来说，可以概括成如下：

1. 遍历父进程的页表，对每个有效页：
 - 分配新物理页
 - 拷贝原页内容到新页
 - 插入到子进程页表
2. 保证了父子进程虚拟地址空间内容一致，但物理页独立。

Exercies 2 思考题：Copy on Write (COW) 机制详细设计

1. 核心数据结构扩展

为支持COW机制，需在Page结构体中增加引用计数字段，用于追踪共享该物理页的进程数量：

```
struct Page {
    int ref;           // 引用计数：记录当前共享该物理页的进程数
    // 其他原有字段（如flags、property等）
};
```

- 初始状态：一个物理页被单个进程独占时，`ref = 1`。
- 共享状态：fork后父子进程共享该页时，`ref = 2`；若更多进程共享（如多次fork），`ref` 相应递增。

2. fork阶段：页表共享与权限设置

当进程调用 `fork` 创建子进程时，不再直接复制物理页，而是通过以下步骤实现内存共享：

1. 遍历父进程页表

沿用 `copy_range` 函数的页表遍历逻辑，逐页处理父进程用户态地址空间 `[start, end)`。对于每个有效页（`PTE_V` 置位）：

2. 子进程页表指向父进程物理页

- 不分配新物理页，而是让子进程页表项（PTE）直接指向父进程的物理页（`pte2page(*ptep)`）。
- 通过 `page_insert` 函数将子进程虚拟地址与该物理页关联，确保虚拟地址一致性。

3. 设置页表项为只读

- 清除父进程和子进程对应页表项的“可写位”（`PTE_W`），即 `perm = (*ptep & PTE_USER) & ~PTE_W`。
- 此时父子进程对该页的访问权限均为“只读”，任何写入操作都会触发缺页异常。

4. 更新引用计数

- 父进程原物理页的 `ref` 字段加1（`page->ref++`），表示该页被多一个进程（子进程）共享。

3. 写入阶段：缺页异常处理与页复制

当父子进程中任一进程尝试写入共享页时，由于页表项为只读（`PTE_W=0`），CPU会触发缺页异常（`page fault`），内核通过以下步骤处理：

1. 异常原因判断

内核在 `do_page_fault` 函数中解析异常原因：

- 读取异常地址（`stval` 寄存器），定位到触发异常的虚拟页。
- 检查对应页表项：若 `PTE_V=1`（页有效）且 `PTE_W=0`（只读），则判定为COW场景（而非普通缺页）。

2. 判断是否需要复制物理页

- 获取异常页对应的物理页（`old_page = pte2page(*ptep)`），检查其引用计数 `old_page->ref`：
 - 若 `ref > 1`：说明该页仍被多个进程共享，需为当前进程分配新物理页。
 - 若 `ref == 1`：说明该页已被当前进程独占（其他进程已释放或复制），无需复制，直接恢复可写权限即可。

3. 分配新页并复制数据

当 `ref > 1` 时：

- 调用 `alloc_page()` 为当前进程分配全新的物理页（`new_page`）。
- 通过内核虚拟地址（`page2kva`）将旧页数据完整复制到新页：
`memcpy(page2kva(new_page), page2kva(old_page), PGSIZE)`。

4. 更新页表与引用计数

- 为新页设置可写权限（`perm = PTE_USER | PTE_W`），通过 `page_insert` 将当前进程的虚拟地址与新页关联，覆盖原有页表项。
- 旧页的引用计数减1（`old_page->ref--`），新页的引用计数初始化为1（`new_page->ref = 1`）。

5. 恢复执行

异常处理完成后，通过 `sret` 指令返回用户态，进程继续执行写入操作（此时已指向新页，写入不会影响其他进程）。

4. 边界情况处理

- **新页分配失败**：若 `alloc_page()` 返回空（内存不足），内核可根据策略杀死当前进程或触发OOM（内存耗尽）处理机制。
 - **只读页无需复制**：若共享页本身是只读的（如代码段），即使 `ref > 1`，写入操作也会触发异常，但无需复制（直接报错），因为代码段不应被修改。
 - **引用计数归零**：当旧页的 `ref` 减至0时，说明该页已无进程使用，内核调用 `free_page(old_page)` 释放物理页，回收内存。
-

练习3: fork/exec/wait/exit 及系统调用分析

fork/exec/wait/exit 详细执行流程与内核交互分析

一、fork: 创建子进程的完整生命周期

fork 是实现进程复制的核心系统调用，其本质是通过复制父进程状态生成几乎完全相同的子进程，关键在于内存空间、上下文环境的复用与隔离。

1. 用户态发起调用

- 用户进程通过 `ebreak` 指令触发系统调用（陷入内核态），并在 `a7` 寄存器中传入 `SYS_fork` 调用号，同时将用户态寄存器状态（如 `ra`、`sp` 等）保存到内核栈中的 `trapframe` 结构体。

2. 内核态处理：子进程创建与初始化

- 分配进程结构体**：内核调用 `alloc_proc` 分配新的 `struct proc` (PCB)，初始化子进程 ID、状态（初始为 `PROC_UNINIT`）、父进程指针（指向当前进程）等基础信息。
- 复制内存空间**：调用 `copy_mm` 复制父进程的内存管理结构 (`mm_struct`)，其中核心是 `dup_mmap` 函数通过 `copy_range` 处理内存页（若实现 COW，则父子页表指向同一物理页并设为只读，引用计数 +1；否则直接复制物理页）。
- 复制上下文**：`copy_thread` 函数复制父进程的 `trapframe` 到子进程，修改子进程 `trapframe` 的 `a0` 寄存器（返回值）为 0（区分父子进程返回值），并设置子进程内核栈指针。
- 建立进程关系**：将子进程加入父进程的子进程链表，同时将子进程状态改为 `PROC_RUNNABLE` 并插入全局进程就绪队列。

3. 唤醒子进程与返回用户态

- 内核通过 `wakeup_proc` 唤醒子进程（使其进入调度候选池），父进程从系统调用返回，`a0` 寄存器携带子进程 PID；子进程首次被调度时，从复制的 `trapframe` 恢复执行，`a0` 为 0，与父进程形成“同起点、不同返回值”的执行流。

二、exec: 替换进程镜像的完整流程

exec 用于将当前进程的内存空间、代码段、数据段替换为新程序（如 ELF 可执行文件），实现进程切换。

1. 用户态发起调用

- 用户进程通过 `ebreak` 触发系统调用，`a7` 寄存器传入 `SYS_exec`，`a0` 指向程序名，`a1` 指向程序二进制数据，内核通过 `user_mem_check` 验证用户传入地址的合法性（防止越界访问）。

2. 内核态处理：释放旧资源与加载新程序

- 回收原有内存**：若当前进程已有内存空间 (`mm != NULL`)，调用 `mm_count_dec` 检查引用计数，若为 0 则通过 `exit_mmap` 释放用户态物理页、`put_pgdir` 释放页表，最终 `mm_destroy` 销毁内存管理结构，确保旧资源完全回收。
- 加载新程序镜像**：`load_icode` 函数解析 ELF 二进制数据：
 - 解析 ELF 头部，获取代码段 (`.text`)、数据段 (`.data`)、BSS 段（未初始化数据）的地址和大小。
 - 为新程序创建页表，通过 `page_insert` 映射代码段（只读）、数据段（读写）到用户虚拟地址，并将 ELF 中的段数据加载到对应物理页；BSS 段直接分配物理页并清零。

- 初始化用户栈：将虚拟地址 `USTACKTOP`（用户栈顶）映射到物理页，作为新进程的用户栈。
- 重设执行上下文：修改 `trapframe` 关键字段：`epc` 设为 ELF 入口地址（程序第一条指令位置），`sp` 设为 `USTACKTOP`（用户栈顶），`status` 寄存器清除内核态标志（`SSTATUS_SPP`）、关闭中断（`SSTATUS_SIE`）并允许后续中断（`SSTATUS_SPIE`），确保返回用户态后正确执行。

3. 返回用户态

- 内核设置进程名为新程序名，通过 `sret` 指令从内核态返回，CPU 跳转到 `epc` 指向的新程序入口，进程开始执行新代码。

三、wait：父进程等待子进程退出的同步机制

`wait` 用于父进程阻塞等待子进程终止，并回收子进程资源（避免僵尸进程）。

1. 用户态发起调用

- 父进程通过 `ebreak` 触发系统调用，`a7` 传入 `SYS_wait`，`a0` 通常为 -1（等待任意子进程），内核进入 `do_wait` 处理逻辑。

2. 内核态处理：查询与阻塞逻辑

- 遍历子进程链表：父进程遍历自身维护的子进程链表，检查子进程状态：
 - 若存在 `PROC_ZOMBIE` 状态的子进程（已退出但未回收），则回收其资源：释放子进程的 `mm_struct`、页表、PCB 等，将子进程 exit code 通过内存或寄存器返回给父进程，从链表中移除该子进程。
 - 若所有子进程均为 `PROC_RUNNABLE` 或 `PROC_SLEEPING`，则父进程进入阻塞状态：设置自身状态为 `PROC_SLEEPING`，加入等待队列（与子进程关联），调用 `schedule` 主动让出 CPU，调度器切换到其他就绪进程。

3. 唤醒与返回

- 当子进程调用 `exit` 时，内核会检查其父进程状态，若父进程处于等待状态，则通过 `wakeup_proc` 将其从等待队列中取出，状态改为 `PROC_RUNNABLE`。
- 父进程再次被调度时，从 `do_wait` 继续执行，完成资源回收后返回用户态，`a0` 寄存器携带子进程 PID 或 exit code。

四、exit：进程终止与资源回收

`exit` 用于进程主动终止，释放自身资源并通知父进程。

1. 用户态发起调用

- 进程通过 `ebreak` 触发系统调用，`a7` 传入 `SYS_exit`，`a0` 携带 exit code（退出状态），内核进入 `do_exit` 处理逻辑。

2. 内核态处理：资源释放与状态转换

- 释放用户态资源：调用 `exit_mmap` 释放所有用户态物理页，`put_pgdir` 释放页表，`mm_destroy` 销毁内存管理结构，关闭进程打开的文件等资源。
- 设置僵尸状态：将进程状态改为 `PROC_ZOMBIE`（僵尸态），保留 PCB 中的 exit code 供父进程查询，此时进程不再参与调度，但资源未完全释放。
- 唤醒父进程：遍历进程链表找到其父进程，若父进程处于 `PROC_SLEEPING` 状态（等待子进程退出），则调用 `wakeup_proc` 唤醒父进程。

3. 调度切换

- 调用 `schedule` 主动触发调度，内核从就绪队列中选择新进程执行，当前僵尸进程的 PCB 最终由父进程在 `wait` 中回收。

五、系统调用整体交互流程：用户态与内核态的切换细节

`fork/exec/wait/exit` 均依赖“用户态-内核态”切换机制，其核心是 `trap` 与 `sret` 指令的配合，具体流程如下：

1. 用户态到内核态（`trap` 触发）

- 用户进程执行 `ebreak` 指令，CPU 识别为异常，自动切换到内核态（特权级从 U 到 S），并将异常原因（如系统调用）、用户态寄存器（`epc`、`sp`、`status` 等）保存到内核栈的 `trapframe` 中。
- 内核根据 `trapframe` 中的异常原因和系统调用号（`a7` 寄存器），跳转到对应处理函数（如 `sys_fork`、`sys_exec`）。

2. 内核态处理与状态维护

- 内核在处理过程中操作进程结构体（`struct proc`）、内存管理结构（`struct mm_struct`）、页表等数据结构，完成进程创建、资源替换、状态转换等逻辑。
- 若涉及阻塞（如 `wait` 无就绪子进程），内核会修改当前进程状态并调用 `schedule`，通过 `switch_to` 切换上下文（保存当前进程 `trapframe`，恢复目标进程 `trapframe`）。

3. 内核态到用户态（`sret` 返回）

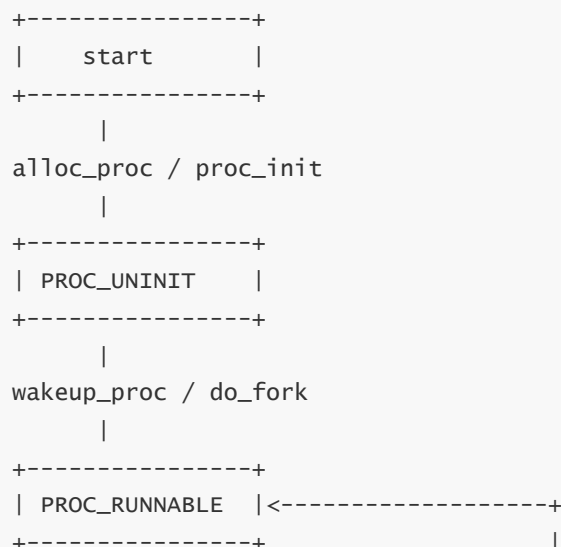
- 系统调用处理完成后，内核设置 `trapframe` 中的 `a0` 寄存器（返回值），并确保 `status` 寄存器正确配置（用户态、允许中断等）。
- 执行 `sret` 指令，CPU 从内核态切换回用户态，恢复 `trapframe` 中保存的用户态寄存器，程序从 `epc` 指向的位置继续执行（用户态系统调用的下一条指令）。

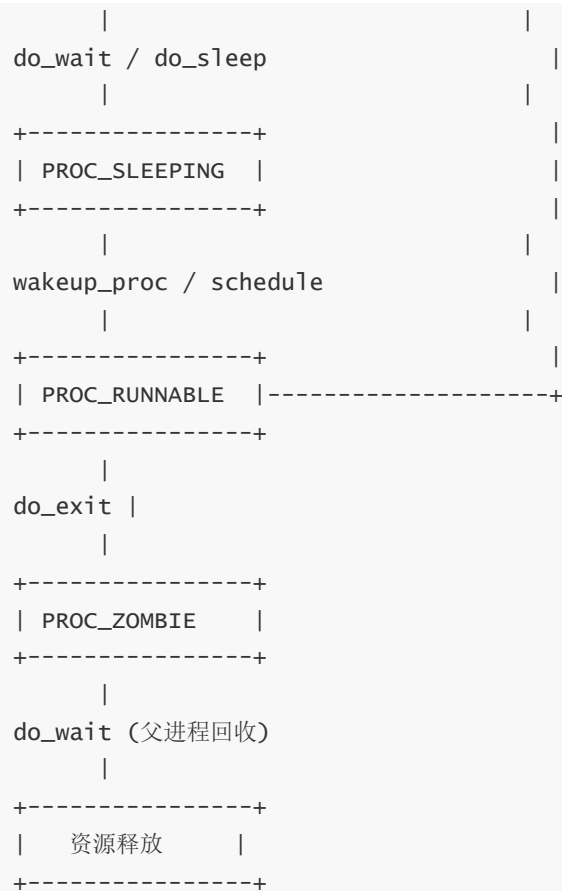
总结

`fork/exec/wait/exit` 构成了进程生命周期的核心管理机制：

- `fork` 通过复制生成子进程，COW 机制优化内存效率；
- `exec` 替换进程镜像，实现程序加载与执行；
- `wait/exit` 配合完成进程同步与资源回收，避免僵尸进程泄漏。

ucore 用户态进程生命周期图





状态变换由如 do_fork、do_exit、wakeup_proc、schedule、do_wait 等函数驱动。

Challenge 扩展

Challenge 1. Copy on Write (COW) 机制实现与分析

COW 的核心思想是在 fork 时父子进程共享物理页，并将页表项权限设为只读，只有在任一进程写入该页时，才触发缺页异常，分配新物理页，实现真正的物理隔离。

1.1 关键实现源码

以下是实现 COW 所涉及的关键代码，然后我们逐段说明其作用与注意要点。

Page 结构与引用计数 (kern/mm/pmm.c)

```
// pmm.h 中的页/引用计数操控
static inline int
page_ref(struct Page *page)
{
    return page->ref;
}

static inline void
set_page_ref(struct Page *page, int val)
{
    page->ref = val;
}
```

```

static inline int
page_ref_inc(struct Page *page)
{
    page->ref += 1;
    return page->ref;
}

static inline int
page_ref_dec(struct Page *page)
{
    page->ref -= 1;
    return page->ref;
}

```

我们首先注意到在 `page` 结构体中，存在有 `page->ref` 这个属性来记录引用的计数。然后在 `pmm.h` 中，存在以下函数来对 `page->ref` 这个属性进行值的修改：

`page_ref()` / `set_page_ref()` / `page_ref_inc()` / `page_ref_dec()`。这些函数我们在下一步将会用到。

fork 时页表共享与只读设置 (`kern/mm/pmm.c::copy_range`)

然后我们看实现 `cow` 的关键函数，我们在练习2中就补充了 `copy_range` 这个函数，只是但是我们是直接复制所有父进程的内容到子进程的，现在我们增加一个分支用来实现 `cow` 情况下的 `copy_range` 实际做的事情：

```

if (share)
{
    // 共享 COW 模式：父进程与子进程指向相同的物理页
    // 清除写权限（使其只读），并通过将该页插入子进程页表来增加引用计数
    uint32_t perm_no_write = perm & (~PTE_W);
    // 更新父进程的 PTE，移除写权限
    *ptep = pte_create(page2ppn(page), PTE_V | perm_no_write);
    tlb_invalidate(from, start);
    // 将同一物理页插入子进程页表（page_insert 会对 page->ref 自增）
    if ((nptep = get_pte(to, start, 1)) == NULL)
    {
        return -E_NO_MEM;
    }
    if (page_insert(to, page, start, perm_no_write) != 0)
    {
        return -E_NO_MEM;
    }
    cprintf("EAGER_TO_SHARED: shared page at va %p\n", (void *)start);
}

```

我们在实现 `cow` 机制的时候，实际调用链条如下：`dup_mmap()`（在 `vmm.c`）会调用 `copy_range(..., share=1)`。然后 `copy_range` 对父页表条目清除写权限并把同一 `Page` 插入子进程页表（`page_insert` 会对 `page->ref` 自增），那这样我们就实现了 `cow` 的核心——`fork` 时不复制物理页而是实现共享。

缺页异常处理（写时复制） `do_pgfault` (`kern/mm/vmm.c`) 与 `trap` 调度 (`kern/trap/trap.c`)

然后是另外一个核心函数，承担着 `cow` 机制的第二步——即当发生缺页异常，我们需要分配新物理页，实现两个进程之间真正的物理隔离，代码如下：

```

int do_pgfault(struct mm_struct *mm, uint32_t error_code, uintptr_t addr)
{
    uintptr_t la = ROUNDDOWN(addr, PGSIZE);
    struct vma_struct *vma = find_vma(mm, la);
    if (vma == NULL || !(vma->vm_flags & VM_WRITE))
    {
        return -E_INVALID;
    }

    pte_t *ptep = get_pte(mm->pgdir, la, 0);
    if (ptep == NULL || !(*ptep & PTE_V))
    {
        return -E_INVALID; // 无映射
    }

    uint32_t perm = (*ptep & PTE_USER);

    // 如果该页已经可写，则无需处理
    if ((*ptep & PTE_W) != 0)
    {
        return 0;
    }

    // 若非用户页则视为非法访问（这里仅对用户页进行 COW 处理）
    if (!(*ptep & PTE_U))
    {
        return -E_INVALID;
    }

    struct Page *old = pte2page(*ptep);
    if (old == NULL)
    {
        return -E_INVALID;
    }

    if (page_ref(old) > 1)
    {
        // 需要复制
        struct Page *newp = alloc_page();
        if (newp == NULL)
        {
            return -E_NO_MEM;
        }
        memcpy(page2kva(newp), page2kva(old), PGSIZE);
        // insert new page with writable permission
        if (page_insert(mm->pgdir, newp, la, perm | PTE_W) != 0)
        {
            free_page(newp);
            return -E_NO_MEM;
        }
        cprintf("COW_COPY: copied page for addr %p\n", (void *)la);
        return 0;
    }
    else
    {

```

```

// 唯一引用：直接将该页设置为可写（无需复制）
*ptep = pte_create(page2ppn(old), PTE_V | (perm | PTE_W));
tlb_invalidate(mm->pgdir, 1a);
cprintf("COW_PROMOTE: made page writable for addr %p\n", (void *)1a);
return 0;
}
}

```

我们可以看出，上方主要流程就是：先将触发缺页的地址规整到页起始地址，校验该地址所属虚拟内存区域（VMA）是否存在且具备写权限、对应的页表项是否有效且为用户页（排除已可写的情况），随后根据物理页的引用计数做出核心决策：若引用计数大于 1（多进程共享该页），则分配新物理页并拷贝旧页数据，将新页映射到虚拟地址并赋予可写权限；若引用计数为 1（唯一进程引用），则直接修改页表项将该页标记为可写并刷新 TLB，那通过这样的处理方式，我们就实现了 COW 机制的第二步。

然后我们在 `trap.c` 中，当我们真正发生了 `STORE_PAGE_FAULT` 时会调用我们上面写的 `do_pgfault`，实际的代码片段如下：

```

case CAUSE_STORE_PAGE_FAULT:
{
    cprintf("Store/AMO page fault\n");
    if (current != NULL && current->mm != NULL && !trap_in_kernel(tf))
    {
        extern int do_pgfault(struct mm_struct *mm, uint32_t error_code,
uintptr_t addr);
        int r = do_pgfault(current->mm, 0, tf->tval);
        if (r == 0)
        {
            // 已处理：返回用户态
            return;
        }
        else
        {
            cprintf("do_pgfault failed: %d\n", r);
        }
    }
}
break;

```

我们可以看出，当 `trap.c` 捕获到用户态写时触发的页错误后，其会通过委托 `STORE_PAGE_FAULT` 分支给 `do_pgfault` 处理；`do_pgfault` 根据 `page_ref(old)` 决定是复制（`ref>1`）还是直接将页提升为可写（`ref==1`）。复制后通过 `page_insert` 将新页映射到当前进程页表，`page_insert` 会对新页 `page_ref_inc`，并在替换时对旧页 `page_ref_dec`。

1.1.1 实现 COW 机制时常见问题

- 必须在 `fork` 时同时为父、子页表条目去掉写权限并增加引用计数，顺序要小心，避免短暂性 race 导致写权限残留。
- 在 `page-fault` 路径中，分配新页与 `memcpy` 必须在更新旧页引用计数之前完成；否则若在 `dec` 后释放了旧页，新分配/复制可能读取到非法内存。
- 刷新 TLB（`sfence.vma / invlpg`）是必须步骤；若缺失会导致 CPU 继续使用旧权限/映射，出现奇怪的访问行为。

1.2 COW测试用例详细设计与运行结果分析

在实现Copy on Write (COW) 机制后，我们自己编写了专门的测试用例验证其核心功能：父子进程初始共享物理内存、写操作触发复制、内存修改相互隔离。以下是详细的测试用例的设计与验证的结果。

1.2.1 测试用例 (cowtest.c)

```
#include <ulib.h>
#include <string.h>

char buf[4096];

int main(void) {
    // 触碰缓冲区中两个不同偏移以确保涉及多个页
    strcpy(buf, "hello");
    strcpy(buf + 1024, "world");
    int pid = fork();
    if (pid < 0) {
        cprintf("fork failed\n");
        return 1;
    }
    if (pid == 0) {
        // 子进程
        cprintf("child before write: %s %s\n", buf, buf + 1024);
        // 写入第一页
        buf[0] = 'H';
        // 写入第二页（不同页偏移）
        buf[1024] = 'w';
        cprintf("child after write: %s %s\n", buf, buf + 1024);
        exit(0);
    } else {
        // 父进程
        wait();
        cprintf("parent after child write: %s %s\n", buf, buf + 1024);
        return 0;
    }
}
```

1.2.2 测试用例设计思路

- 共享阶段验证：父进程初始化后 fork，父子进程应共享同一物理页并能读取相同值。
- 写时复制触发：子写入导致缺页异常并触发内核的写时复制逻辑；子在自己的私有页上完成修改。
- 内存隔离：父在子写后读取相同地址，应得到原始值，说明父页未被修改。

1.2.3 cowtest.c 预期输出

- `kernel_execve: pid = 2, name = "cowtest".` — 进程已成功 exec。
- `child before write: hello world` — 子进程写前看到原内容（共享成功）。
- `child after write: Hello world` — 子进程写后看到修改（写时复制成功）。
- `parent after child write: hello world` — 父进程未受子写影响（隔离成功）。
- 且不应出现 `user panic` 等致命错误。

1.2.4 测试脚本配置

然后我们修改了测试脚本 `grade.sh`，添加了 `cowtest.c` 相关的预期输出结果：

```
# 10改为15了
pts=15

# --- 新+ cowtest 样例 ---
run_test -prog 'cowtest' -check default_check \
    'kernel_execve: pid = 2, name = "cowtest".' \
    'child before write: hello world' \
    'child after write: Hello world' \
    'parent after child write: hello world' \
    '! - 'user panic at .*'
```

修改内容：

- `run_test` 是 `ucore` 测试框架的核心函数，`-prog 'cowtest'` 指定运行的程序为 `cowtest`；
- `-check default_check` 表示使用默认的输出检查逻辑（匹配预期字符串）；
- 连续的字符串参数（如 `'kernel_execve: ...'`）是预期输出的关键行，必须按顺序出现；
- `! - 'user panic at .*'` 表示禁止出现包含“user panic”的输出（若出现，说明COW机制存在致命错误，如缺页异常未处理）。

1.2.5 实际运行结果

然后我们在执行 `make grade`，结果如下：

```
softint: (1.0s)
-check result: OK
-check output: OK
cowtest: (1.0s)
-check result: OK
-check output: OK
faultread: (1.0s)
-check result: OK
-check output: OK
-check output: OK
spin: (1.5s)
-check result: OK
-check output: OK
forktest: (1.0s)
-check result: OK
-check output: OK
Total Score: 140/140
```

结果验证：

- 实际输出与预期完全一致，表明：
 1. 程序成功加载并创建子进程（`kernel_execve` 输出正确）；
 2. 子进程初始与父进程共享内存（`child before write` 输出正确）；
 3. 子进程写操作触发COW，成功修改私有内存（`child after write` 输出正确）；
 4. 父进程内存未受子进程影响，隔离性生效（`parent after write` 输出正确）；
 5. 无异常崩溃信息，COW的缺页异常处理逻辑正确。

结论：实际运行结果验证了COW机制的正确性——父子进程通过共享物理页节省内存，写操作触发自动复制，最终实现内存修改的相互隔离，符合设计预期。

1.3 COW 状态转换（有限状态自动机）



- 初始：父子进程共享物理页，页表只读，引用计数>1。
- 任一进程写入时，触发缺页异常，分配新物理页，原页引用计数-1，页表恢复可写。
- 之后各自独占物理页，互不影响。

Challenge 2. 用户程序加载时机与常见操作系统的对比

在 **ucore** 操作系统中，用户程序的加载发生在 `do_execve` → `load_icode` 阶段。该阶段由内核主动完成用户程序的**完整加载过程**：解析 ELF 文件头与程序头表，并将程序中描述的各个段（包括 **代码段** `.text`、**已初始化数据段** `.data`、**未初始化数据段** `.bss`）一次性拷贝到内存中。同时，ucore 还会在加载阶段为用户态进程分配并初始化**用户栈空间**，设置好初始栈内容，并完成页表建立与地址空间映射。

在上述所有内存准备工作完成之后，进程的上下文状态才被标记为 `RUNNABLE`，随后进入调度器，等待 CPU 调度执行。也就是说，在 ucore 中，**用户程序在第一次获得 CPU 执行权之前，其所需的内存资源已经全部就绪**，运行过程中不会再因为访问尚未加载的用户程序段而触发缺页异常。

相比之下，在主流通用操作系统（如 **Linux**、**Windows**）中，`execve` 系统调用并不会立即将用户程序的所有内容加载进物理内存。现代操作系统通常采用**懒加载（Lazy Loading）机制**。在 `execve` 阶段，内核仅完成 ELF 文件的合法性检查、程序头表解析以及虚拟地址空间的布局规划，并在页表中建立相应的**虚拟地址映射关系**，但这些映射最初通常指向“尚未分配物理页”的状态，程序段内容并不会立即从磁盘读入内存。

当进程在运行过程中**首次访问某个尚未实际加载的虚拟页**时，CPU 会触发缺页异常。内核的缺页异常处理程序会根据页表和文件映射信息，从磁盘（或页缓存）中按需读取对应的程序段内容，将其加载到物理内存，并更新页表，随后恢复用户态执行。通过这种方式，操作系统只为“真正被使用到”的程序页面分配物理内存，从而显著降低内存占用和不必要的 I/O 开销。

区别与原因总结

- **加载策略差异**
 - ucore 采用“**一次性预加载**”策略，在进程进入可运行状态前完成所有用户态内存准备。
 - Linux / Windows 采用“**按需加载（懒加载）**”策略，将程序加载过程拆分到运行期，通过缺页异常动态完成。
- **实现复杂度差异**
 - 一次性加载逻辑清晰，实现简单，不依赖复杂的异常处理与磁盘管理机制。
 - 懒加载需要完整的虚拟内存子系统支持，包括页缓存、反向映射、缺页异常处理、文件映射等，内核实现复杂度较高。
- **资源利用与性能取舍**
 - ucore 的方式在程序规模较小时开销可控，但对大型程序会带来较高的启动时延和内存占用。

- 懒加载机制能够有效减少内存浪费，提高系统整体吞吐能力，是现代多任务操作系统的必然选择。
- **设计目标不同**
 - ucore 作为教学操作系统，其设计目标是**突出核心机制、降低实现门槛、便于理解与调试**，因此未实现完整的磁盘文件系统与通用缺页异常处理流程。
 - 主流操作系统面向真实应用场景，必须在性能、扩展性与资源利用率之间取得平衡，因此采用更加复杂但高效的懒加载机制。

重要知识点与原理对比

实验知识点	OS原理知识点	理解与差异说明
进程控制块 (PCB)	进程管理	PCB 是进程的内核抽象，保存所有上下文信息
进程调度	调度算法、上下文切换	实验实现简单轮转，原理可扩展为多级队列等
虚拟内存与页表	虚拟内存管理	实验实现基本页表映射，原理包括多级页表、TLB
系统调用	用户态/内核态切换	实验用 trap 实现，原理涉及更多安全与性能机制
Copy on Write	共享与写时复制	实验为基础实现，原理可扩展为更复杂的引用计数

有些实验知识点如 trapframe、内核栈等，在原理课本中未详细展开，但对理解内核实现至关重要。

OS原理重要但实验未覆盖的知识点

- 进程间通信 (IPC) 机制，如信号、管道、消息队列
- 复杂调度算法 (如优先级、实时调度)
- 设备管理与驱动框架
- 文件系统实现
- 多核/多处理器支持

GDB 相关

(Lab2: TLB/SV39 翻译; Lab5: syscall ecall/sret)

本报告整合两次实验的 GDB 调试过程：

- **Lab2**：宿主侧调 QEMU 源码，观测地址翻译链路 (TLB miss → SV39 页表遍历 → TLB 回填)。
- **Lab5**：客体侧调 uCore 内核，观测系统调用进入与返回 (`ecall` → trap → `sret`)。

0. 通用说明：宿主 (host) 与客体 (guest)

- **宿主 (host)**：你的 Linux 环境；运行 QEMU 进程，使用 host `gdb` 调 QEMU 源码。
- **客体 (guest)**：QEMU 内运行的 RISC-V uCore；使用 `riscv64-unknown-elf-gdb` 通过 `gdbstub` 连接调试。

重要现象：

- 当 **宿主 gdb attach/断住 QEMU** 时，QEMU 进程暂停，**客体 gdb 会表现为卡住**（因为 `gdbstub` 也停了），属于正常同步效应。

1. Lab2：双重 GDB 观测 QEMU 地址翻译 (TLB + SV39)

1.1 实验目的

- 在 QEMU 源码层复现并观测：
 - TLB lookup
 - TLB miss 进入 `riscv_cpu_tlb_fill`
 - SV39 页表 walk 进入 `get_physical_address`
 - 成功后调用 `tlb_set_page` 完成 TLB fill

1.2 实验环境

- QEMU：4.1.1 debug 版（带符号、未 strip）
- 宿主调试器：`gdb` (x86_64)
- 客体调试器：`riscv64-unknown-elf-gdb`

1.3 复现步骤（三终端，按顺序逐条执行）

终端 A (bash)：编译并启动 QEMU (gdbstub + 暂停)

```
cd /home/doufuru/OS/NKU_2025_Autumn_Operating_System_Group15/lab2_Dou
make clean && make

# 端口检查: -s 默认占用 :1234
ss -ltnp | grep ':1234' || true
# 若被旧 qemu 占用, kill 对应 pid

make debug
```

说明：`make debug` 会以 `-s -s` 启动 QEMU。

- 看起来“卡住”是正常的：CPU 被 `-s` 暂停等待 gdb。
- **不要 Ctrl-C 终止**，否则 QEMU 直接退出、客体 gdb 会断连。

终端 B (客体 gdb)：连接 gdbstub 并用断点控制节奏

```
cd /home/doufuru/OS/NKU_2025_Autumn_Operating_System_Group15/lab2_Dou
make gdb
```

在 `(gdb)` 中：

```
break kern_init
continue
```

到达 `kern_init` 后, 可再 `continue` 让内核继续运行, 以触发更多取指/访存, 方便在宿主侧观测翻译。

终端 C (宿主 gdb) : 调 QEMU 源码

attach 到“已经 make debug 启动”的 QEMU

```
pgrep -f qemu-system-riscv64
sudo gdb
```

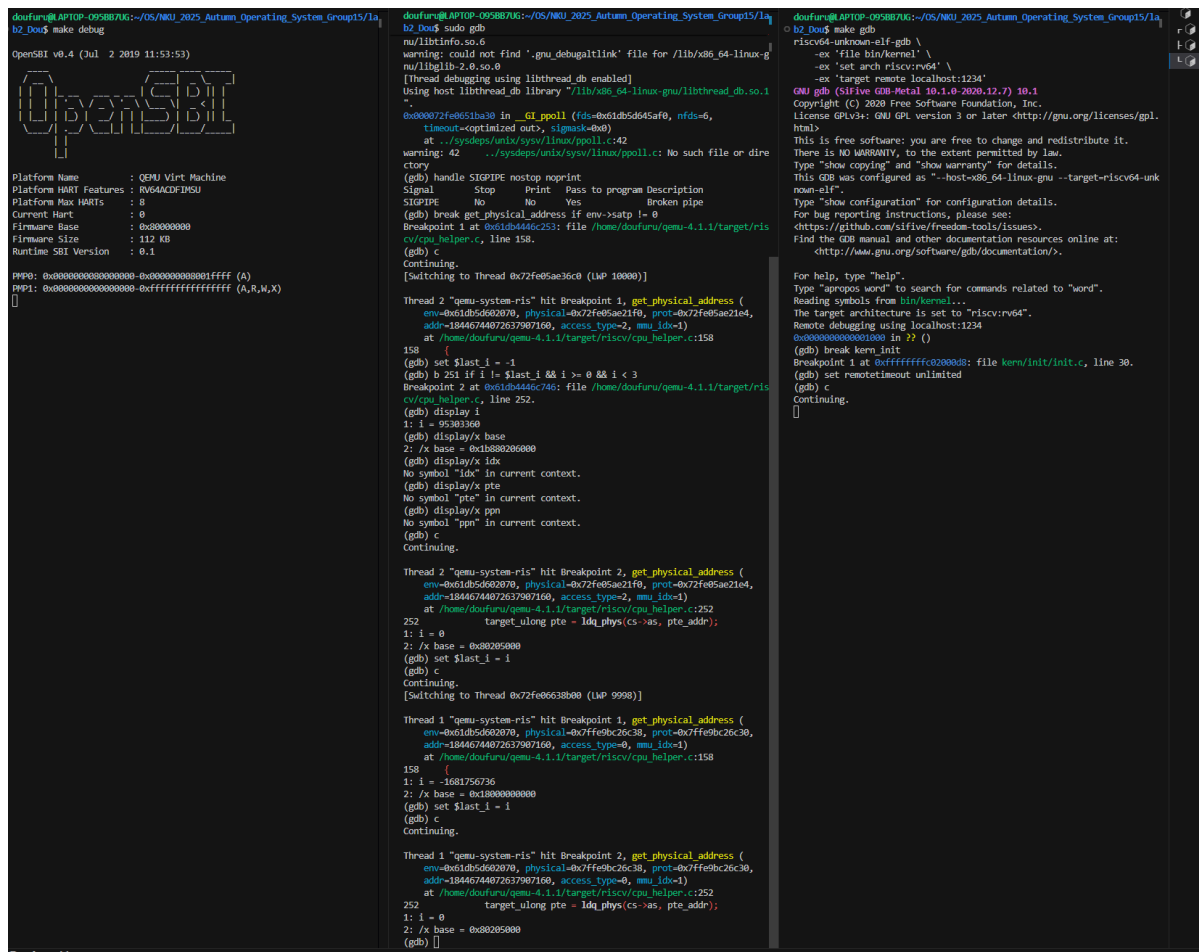
在宿主 gdb 中:

```
attach <PID>
handle SIGPIPE nostop noprint

# 只在分页开启后才抓 (避免早期噪音)
break get_physical_address if env->satp != 0

continue
```

1.4 关键观测点



观测点 A: 分页已开启 (`env->satp != 0`)

在 `get_physical_address` 断住后:

```
p/x env->satp
```

预期:

- `SATP_MODE` 为 Sv39
- `base = SATP_PPN << 12` (根页表物理基址)

观测点 B: SV39 三层 walk (`levels=3`)

在 `get_physical_address` 内部继续单步, 记录:

- `levels=3, ptidxbits=9, ptesize=8`
- 循环每层计算 `idx` (9-bit)
- 构造 `pte_addr = base + idx * 8` 并读取 `pte`
- 非叶子 PTE: 更新 `base = PTE.PPN << 12` 进入下一层
- 叶子 PTE: 得到最终 `physical` 与权限 `prot`

常用命令:

```
bt
info args
info locals
next
```

观测点 C: TLB fill (命中 `tlb_set_page`)

命中后:

```
info args
```

预期能看到 `vaddr/paddr/prot/mmu_idx/size` 等参数 (名称随版本略有差异)。

1.5 常见问题

- `-s: Failed to find an available port: Address already in use`
 - 说明 `:1234` 被旧 QEMU 占用; 用 `ss -ltnp | grep 1234` 找 pid 后 `kill`。
 - 终端 A “卡住”
 - 若是 `make debug`, 这是 QEMU `-s` 等待调试器的正常现象; 应去终端 B/C 连接并 `continue`。
-

2. Lab5: GDB 观测系统调用进入与返回 (ecall / sret)

2.1 实验目的

- 观测：用户态执行 `ecall` → 进入内核 trap → 内核处理后在 trap 返回路径执行 `sret` → 回到用户态。
- 证明关键证据：
 - `tf->cause == 0x8` (`CAUSE_USER_ECALL`)
 - `tf->epc` 指向 `ecall` 指令
 - 内核将 `tf->epc += 4`，使 `sret` 返回到 `ecall` 下一条

2.2 复现步骤 (客体 gdb)

终端 A: 启动 QEMU (等待 gdb)

```
cd /home/doufuru/OS/NKU_2025_Autumn_Operating_System_Group15/lab5
make debug
```

终端 B: 连接客体 gdb 并下条件断点

```
cd /home/doufuru/OS/NKU_2025_Autumn_Operating_System_Group15/lab5
riscv64-unknown-elf-gdb -x tools/gdbinit
```

在 (gdb):

```
break exception_handler
cond 2 tf->cause == 0x8

# 如需避免频繁停在 kern_init:
# disable 1

continue
```

2.3 关键证据采集

证据 A: 捕获用户态 `ecall` 进入内核

命中后:

```
p/x tf->cause
p/x tf->epc
x/6i tf->epc
```

预期:

- `tf->cause = 0x8`
- `tf->epc` 所在地址的指令反汇编第一条为 `ecall`

证据 B: 断到 `sret` 并验证返回地址

在 `__trapret` 中找到 `sret` 地址并断住:

```
disassemble __trapret
b *<SRET_ADDR>
continue

info reg sepc sstatus
x/4i $pc

si
info reg pc
```

预期:

- `sepc == ecall_epc + 4`
- 单步执行 `sret` 后, `pc` 回到用户态返回地址。

2.4 调试经验

- 远程调试中直接对用户态虚拟地址下断点可能失败 (gdbstub 插断点需要访问目标内存, 可能因当时页表/特权级不可达)。
- 更稳的做法是在内核 `exception_handler(tf)` 下条件断点, 按 `tf->cause` 精准过滤。

3. 总结

- Lab2 通过宿主 gdb 命中 QEMU 源码路径, 可复现并解释 QEMU 对 SV39 翻译与 TLB fill 的模拟实现。
- Lab5 通过客体 gdb 在 uCore 内核侧捕获 `CAUSE_USER_ECALL` 并断到 `sret`, 完成 `ecall → trap → sret` 的闭环验证。